

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
"РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ имени А. И. ГЕРЦЕНА"



Направление подготовки

09.03.01 Информатика и вычислительная техника

Направленность

"Технологии разработки программного обеспечения и обработки больших
данных"

Курсовая работа

**Разработка и развертывание веб приложения для учета
документов в МЧС с использованием Docker Compose и
автоматической сертификации через Let's Encrypt + ACME
Companion**

Выполнил: студент 1об_ИВТЗ
Карандашов Артемий Александрович

Проверил(а):

Санкт-Петербург

2026

ОГЛАВЛЕНИЕ.

ВВЕДЕНИЕ.....	3
ГЛАВА 1.ТЕОРЕТИЧЕСКИЕ ОСНОВЫ КОНТЕЙНЕРИЗАЦИИ И АВТОМАТИЧЕСКОЙ СЕРТИФИКАЦИИ ВЕБ-ПРИЛОЖЕНИЙ.....	5
1.1. КОНТЕЙНЕРИЗАЦИЯ И DOCKER КАК ОСНОВА СОВРЕМЕННОЙ РАЗВЕРТКИ ПРИЛОЖЕНИЙ.....	5
1.2. МНОГО КОНТЕЙНЕРНЫЕ ПРИЛОЖЕНИЕ С DOCKER COMPOSE.....	7
1.3. ОБЕСПЕЧЕНИЕ БЕЗОПАСНОЙ ПЕРЕДАЧИ ДАННЫХ.....	8
1.4. АВТОМАТИЧЕСКАЯ СЕРТИФИКАЦИЯ ПРИ ПОМОЩИ LET'S ENCRYPT И ACME COMPANION.....	10
1.5. ВЫВОДЫ.....	12
ГЛАВА 2. РАЗРАБОТКА АВТОМАТИЧЕСКОГО РАЗВЕРТЫВАНИЯ ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ ДОКУМЕНТООБОРОТА В МЧС.....	13
2.1 ПОДГОТОВКА DOCKERFILE ДЛЯ КОНТЕЙНЕРИЗАЦИИ ПРИЛОЖЕНИЯ.....	13
2.2 ПОДГОТОВКА DOCKER-COMPOSE С АВТОМАТИЧЕСКОЙ ВЫПИСКОЙ SSL/TLS СЕРТИФИКАТОВ.....	16
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	22

ВВЕДЕНИЕ.

В условиях цифровизации государственных структур, в том числе МЧС, возникает необходимость в быстрых и удобных сопровождающих веб-сервисом. К таким сервисам, в частности, относятся системы учёта входящих документов, контроля исполнительской дисциплины и внутреннего документооборота. Традиционные подходы к развертыванию веб-приложений предполагают ручную настройку веб-сервера (Nginx, Apache), установку зависимостей на целевой сервер, настройку SSL-сертификатов и последующее периодическое обновление этих сертификатов. Такой подход трудоемок, подвержен рискам ошибок и плохо масштабируется при появлении новых сервисов.

Контейнеризация и оркестрация приложений позволяют упаковать приложение со всеми его зависимостями в изолированные контейнеры и обеспечить воспроизводимость окружения на любом сервере. Однако даже при использовании контейнеров остаётся проблема автоматической настройки HTTPS и управления SSL-сертификатами. Let's Encrypt предлагает бесплатные сертификаты, а протокол ACME - механизм их автоматического получения и продления. В связке с Docker Compose удобным решением является использование nginx-proxy и acme-companion, которые динамически настраивают обратный прокси и сами запрашивают сертификаты для новых контейнеров. Таким образом, лишь раз развернув приложение одной простой командой, оно само будет обновлять истекшие сертификаты и не потребует дальнейшего сопровождения в этой сфере.

Таким образом, разработка и развёртывание веб-приложения с применением указанных инструментов представляет собой актуальную задачу, особенно для организаций с ограниченным количеством сотрудников ИТ

сферы, где автоматизация и сокращение ручного вмешательства критически важны.

Таким образом, цель данной работы - разработка автоматизированного развертывания веб-приложения учёта документов с использованием Docker Compose, а также интеграция автоматической сертификации через Let's Encrypt и ACME Companion.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ КОНТЕЙНЕРИЗАЦИИ И АВТОМАТИЧЕСКОЙ СЕРТИФИКАЦИИ ВЕБ-ПРИЛОЖЕНИЙ.

1.1. КОНТЕЙНЕРИЗАЦИЯ И DOCKER КАК ОСНОВА СОВРЕМЕННОЙ РАЗВЕРТКИ ПРИЛОЖЕНИЙ.

Современная разработка программного обеспечения сталкивается с проблемой несовместимости окружений: код, работающий на компьютере разработчика, может отказаться функционировать на сервере из-за различий в версиях операционной системы, библиотек и системных утилит. Одно из решений - использование виртуальных машин, но оно связано с большими накладными расходами ресурсов, так как каждая виртуальная машина содержит свою операционную систему целиком.

Контейнеризация предлагает альтернативный подход. Контейнер - это изолированное пользовательское пространство, которое использует ядро хостовой операционной системы, но имеет собственную файловую систему, процессы и пользователей. Это позволяет запускать несколько контейнеров на одном хосте с минимальными накладными расходами.

Docker стал фактическим стандартом контейнеризации. Он предоставляет экосистему инструментов: клиент-серверную архитектуру, механизм образов и слоев. Образ - это шаблон для создания контейнера, который строится на основе слоев инструкций. Слои кэшируются, что ускоряет пересборку.

В контексте данной курсовой работы Docker выбран потому, что:

- Он позволяет унифицировать окружение для всех компонентов сервиса документооборота МЧС.
- Процесс развертывания сводится к одной команде.

- Предоставляет возможности как для горизонтального, так и для вертикального расширения сервиса в будущем.

Docker - это, безусловно, стандарт в мире контейнеризации, но далеко не единственное решение. Сейчас существует множество инструментов, каждый из которых предлагает свои уникальные преимущества: от повышенной безопасности до глубокой интеграции с Kubernetes (В этой работе он не рассматривается, но Kubernetes это ПО для оркестровки контейнеризированных приложений).

Ниже представлена сравнительная таблица наиболее популярных альтернатив Docker, основанная на актуальных данных за 2025-2026 годы.

Таблица №1. Сравнительная таблица альтернатив Docker.

Характеристика	Docker	Podman	Containerd	CRI-O	LXC/LXD	BuildKit	Kaniko	Rancher Desktop
Тип / Роль	Полноценная платформа контейнеризации	Безагентный рантайм для контейнеров	Базовый OCI-рантайм от CNCF	Рантайм, созданный для Kubernetes	Системные контейнеры (легковесные VM)	Современный инструмент для сборки образов	Инструмент для сборки образов в Kubernetes	Десктопная альтернатива с K8s
Архитектура	Клиент-серверная с центральным демоном (dockerd)	Без демона, использует fork/exec	Легковесный демон	Легковесный демон для K8s	Демон (LXD) для управления системными контейнерами	Инструмент сборки (не рантайм)	Инструмент сборки (не рантайм)	Десктопное приложение с K3s и Docker/containerd

Rootless	Требуется root по умолчанию	Ключевая особенность - нативная поддержка rootless	Поддерживает	Безопасен по дизайну	Частичная поддержка	Частичная поддержка	Поддерживает	Да
Совместимость с Docker CLI	Полная	Высокая - команды почти идентичны	Частичная (через nerdctl)	Собственный интерфейс (CRI)	Нет	Частичная (сборка Dockerfile)	Нет	Высокая
Основное применение	Разработка, тестирование, небольшие продакшн-среды	Безопасные продакшн-среды, замена Docker	Kubernetes (стандартный рантайм)	Kubernetes (легкий и безопасный рантайм)	Запуск полных ОС, изоляция на уровне системы	CI/CD для высокопроизводительной сборки	CI/CD в облачных средах (Kubernetes)	Замена Docker Desktop с встроенным Kubernetes
Платформы	Linux, Windows, macOS	Linux, Windows (через WSL2)	Linux, Windows, macOS	Linux	Linux	Linux, Windows	Linux, Kubernetes	Linux, Windows, macOS
Лицензия	Apache 2.0 (Engine)	Apache 2.0	Apache 2.0	Apache 2.0	LGPLv2.1+ / Apache 2.0	Apache 2.0	Apache 2.0	Открытое ПО

1.2. МНОГО КОНТЕЙНЕРНЫЕ ПРИЛОЖЕНИЕ С DOCKER COMPOSE.

При разработке реального веб-приложения редко удаётся ограничиться одним контейнером. Состав проекта рассмотренного в данной курсовой работе включает:

- Контейнер с бэкендом (FastAPI);
- Контейнер с базой данных (PostgreSQL);
- Контейнер с кэшем (Redis);
- Прокси контейнер (nginx);
- Контейнер с фронтендом (React).

Управлять каждым контейнером по отдельности - неудобно. Требуется вручную создавать сеть, пробрасывать порты, монтировать тома, указывать зависимости запуска. Docker Compose решает эту проблему. Это инструмент оркестрации, который использует декларативный файл в формате `yaml` для описания всех сервисов, сетей и томов.

В данной работе Docker Compose играет центральную роль: именно он объединяет все компоненты приложения, настраивает сетевые связи и управляет жизненным циклом контейнеров.

1.3. ОБЕСПЕЧЕНИЕ БЕЗОПАСНОЙ ПЕРЕДАЧИ ДАННЫХ.

Передача данных по протоколу HTTP в открытом виде создает серьезные угрозы: перехват трафика, подмена данных, кража учетных данных и документов. Для систем документооборота МЧС защита канала связи является обязательным требованием, так как внутренние приказы, отчеты и персональные данные могут стать объектом атаки.

SSL/TLS (Secure Sockets Layer / Transport Layer Security) - криптографические протоколы, обеспечивающие шифрование передаваемых данных, аутентификацию сервера и целостность сообщений. Работа строится на основе сертификата X.509, который удостоверяет принадлежность публичного ключа определенному домену. Сертификат подписывается центром сертификации - доверенной третьей стороной.

Традиционные коммерческие центры сертификации (Например Sectigo, DigiCert и GlobalSign) выдают сертификаты на платной основе, часто на длительный срок. Однако с 2016 года широкое распространение получил Let's Encrypt - некоммерческий центр сертификации, предоставляющий сертификаты бесплатно, но с коротким сроком действия в 90 дней. Короткий срок стимулирует автоматизацию продления, что повышает безопасность.

Протокол ACME (Automatic Certificate Management Environment) - стандарт, разработанный IETF, который автоматизирует взаимодействие между клиентом и центром сертификации. ACME определяет набор сообщений для запроса, проверки владения доменом, получения и отзыва сертификатов. Типичный процесс:

1. Клиент генерирует ключевую пару и отправляет запрос на создание аккаунта в центр сертификации.

2. Клиент доказывает владение доменом одним из методов: HTTP-01 (размещение файла по известному URL), DNS-01 (публикация TXT-записи), TLS-ALPN-01 (специальный сертификат на порту 443).
3. После успешной верификации центр сертификации выдает запрошенный сертификат.
4. Клиент периодически связывается с центром сертификации для продления сертификата.

Для веб-приложений наиболее удобен метод HTTP-01, так как он не требует прав на управление DNS-зонами и работает через стандартный веб-сервер. Именно его мы будем использовать в данной работе через `acme-companion`.

1.4. АВТОМАТИЧЕСКАЯ СЕРТИФИКАЦИЯ ПРИ ПОМОЩИ LET'S ENCRYPT И ACME COMPANION.

Хотя протокол ACME сам по себе автоматизирует получение сертификатов, интеграция его в окружение Docker требует дополнительных усилий. Вручную запускать certbot на хосте и затем подкладывать сертификаты в контейнеры - не элегантное решение, нарушающее принцип “всё декларативно” и требующее дополнительных скриптов для автоматизации на хосте.

Для бесшовной интеграции с Docker Compose существует несколько решений:

- Traefik - современный обратный прокси с поддержкой ACME из коробки.
- Caddy - веб-сервер с автоматическим HTTPS.
- nginx-проху + acme-companion - классическая связка, состоящая из двух контейнеров.

В данной работе выбран nginx-проху + acme-companion по следующим причинам:

- Разделение ответственности: nginx-проху отвечает за маршрутизацию запросов, acme-companion - за сертификаты.
- Большое сообщество и стабильность.
- Простота настройки: достаточно добавить переменные окружения к контейнеру с приложением.
- Поддержка email уведомлений.

Принцип работы связки в Docker Compose:

1. Контейнер nginx-proxu запускается с монтированием сокета Docker. Он отслеживает появление и исчезновение контейнеров, анализирует их переменные окружения и динамически генерирует конфигурацию Nginx.
2. Контейнер асме-companion также имеет доступ к сокету Docker. При обнаружении нового контейнера с переменной LETSENCRYPT_HOST он пытается получить сертификат через Let's Encrypt, используя метод HTTP-01.
3. Для проверки владения доменом асме-companion просит nginx-proxu временно обслужить специальный временный файл. После успешной верификации сертификат сохраняется в общий том.
4. асме-companion обновляет конфигурацию nginx-proxu, чтобы тот использовал полученный сертификат для HTTPS.
5. Автоматическое продление: асме-companion периодически (каждый день) проверяет срок действия сертификатов и при необходимости инициирует их обновление, после чего перезагружает Nginx.

1.5. ВЫВОДЫ.

В теоретической главе были рассмотрены ключевые концепции, лежащие в основе развертывания веб-приложения документооборота для МЧС:

- Контейнеризация позволяет упаковать каждый компонент в изолированные, легковесные и воспроизводимые окружения.
- Docker Compose дает возможность описать весь стек сервисов декларативно, автоматизировать порядок запуска и связи между контейнерами.
- Протокол SSL/TLS и сертификаты, особенно бесплатные от Let's Encrypt, необходимы для защиты трафика.
- Протокол ACME автоматизирует получение и обновление сертификатов.
- Связка nginx-proxy и acme-companion интегрирует ACME в Docker Compose, требуя лишь указания доменного имени и email.

На основе этих теоретических положений в следующей главе будет реализовано практическое развёртывание демонстрационного сервиса с использованием описанных инструментов.

ГЛАВА 2. РАЗРАБОТКА АВТОМАТИЧЕСКОГО РАЗВЕРТЫВАНИЯ ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ ДОКУМЕНТООБОРОТА В МЧС.

2.1 ПОДГОТОВКА DOCKERFILE ДЛЯ КОНТЕЙНЕРИЗАЦИИ ПРИЛОЖЕНИЯ.

Для демонстрации рассмотренных в первой главе работы возможностей инструментов развертки и сопровождения было выбрано демонстрационное приложение разработанное на 2 курсе обучения как итоговая работа по дисциплине “Программирование”. Данное приложение включает в себя несколько компонентов, а именно фронтенд и бэкенд сервисы, базу данных и кэш сервер. В ходе данной работы было автоматизировано развертывание всего сервиса, а также была добавлена автоматическая сертификация при помощи связки Let’s encrypt и acme-companion.

На первом этапе было необходимо подготовить оптимальные докер-файлы для каждого из будущих контейнеров. Официальные образы Postgres, Redis, nginx-проху и acme-companion уже оптимизированы для развертывания, поэтому остаются только контейнеры самого приложения, а именно фронтенд и бэкенд.

Бэкенд использует множество относительно тяжелых зависимостей, таких как например FastApi и SQLAlchemy, поэтому есть смысл оптимизировать их установку при помощи встроенной утилиты python wheels. Таким образом, построение образа будет разбито на два этапа - установка зависимостей через wheels (Builder) и запуск бэкенд сервиса через unicorn (Runner). Итоговый Dockerfile выглядит следующим образом.

Листинг №1. Dockerfile-backend

```
FROM python:3.13-slim AS builder

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    g++ \
    make \
    python3-dev \
    libffi-dev \
    libssl-dev \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .

RUN pip install --no-cache-dir --upgrade pip wheel setuptools && \
    pip wheel --no-cache-dir --wheel-dir=/wheels -r requirements.txt

FROM python:3.13-slim

WORKDIR /app

COPY --from=builder /wheels /wheels

COPY --from=builder /app/requirements.txt .

RUN pip install --no-cache-dir --find-links=/wheels -r requirements.txt && \
    rm -rf /wheels

COPY . .

ENV SERVER_PORT=8000

EXPOSE ${SERVER_PORT}

CMD uvicorn src.__init__:app --reload --host 0.0.0.0 --port $SERVER_PORT \
    --log-level debug
```


Также, нужно разработать Dockerfile и для фронтэнда. Однако, фронтэнд данного сервиса реализован через React+Typescript, а nginx-проху работает со статической директорией “/usr/share/nginx/html”, значит придется собирать статику с помощью команды `npm run build`. Таким образом итоговый Dockerfile выглядит следующим образом:

Листинг №2. Dockerfile-frontend

```
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install --production=false
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

2.2 ПОДГОТОВКА DOCKER-COMPOSE С АВТОМАТИЧЕСКОЙ ВЫПИСКОЙ SSL/TLS СЕРТИФИКАТОВ.

Теперь, когда для всех образов выбраны или подготовлены докер-файлы, мы можем начать создавать docker-compose файл, который будет развертывать все необходимые для работы приложения контейнеры разом.

Сразу добавим конфигурации для Postgres и Redis, они не требуют особой конфигурации, за исключением задания настроек и портов, на которых они работают. Контейнер server с FastApi имеет переменную среды JWT_SECRET. Её значение должно быть hex строкой произвольной длины, которую ни в коем случае нельзя показывать, ведь она отвечает за безопасность выписанных access и refresh токенов приложения. Теперь нужно правильно настроить контейнер с фронтом. Для корректной работы связки Let's Encrypt + acme companion необходимо добавить две переменные среды: VIRTUAL_HOST и LETSENCRYPT_HOST. В них хранится домен веб-сервиса, для данного проекта, значения этих переменных совпадают. Наконец, когда все настройки были выставлены можно добавить в конфигурацию два контейнера nginx-proxy и acme-companion. Этот фрагмент взят без каких либо изменений из примеров официального репозитория nginx-proxy.

```
services:

  nginx-proxy:

    image: nginxproxy/nginx-proxy:latest

    container_name: nginx-proxy

    ports:

      - "80:80"

      - "443:443"

    volumes:

      - /var/run/docker.sock:/tmp/docker.sock:ro

      - certs:/etc/nginx/certs:ro

      - vhost:/etc/nginx/vhost.d

      - html:/usr/share/nginx/html

    labels:

      - "com.github.jrcs.letsencrypt_nginx_proxy_companion.nginx_proxy"

    restart: unless-stopped

  acme-companion:

    image: nginxproxy/acme-companion:latest

    container_name: nginx-proxy-acme

    volumes:

      - /var/run/docker.sock:/var/run/docker.sock:ro

      - certs:/etc/nginx/certs:rw

      - vhost:/etc/nginx/vhost.d:rw

      - html:/usr/share/nginx/html:rw

      - acme:/etc/acme.sh

    environment:
```

```

    - DEFAULT_EMAIL=artemij.karandashov@yandex.ru

depends_on:
    - nginx-proxy

restart: unless-stopped

client:

    build:

        context: ./client

    container_name: client

    expose:

        - "80"

    environment:

        - VIRTUAL_HOST=mes-db.knuckles.us
        - LETSENCRYPT_HOST=mes-db.knuckles.us

    restart: always

server:

    build: ./server

    container_name: server

    environment:

        - DB_URL=postgresql+asyncpg://postgres:root@postgres:5433/mes_db
        - REDIS_HOST=redis
        - REDIS_PORT=6677
        - JWT_SECRET="SECRET"
        - JWT_ALGORITHM=HS256
        - SERVER_PORT=8000

    ports:

```

```

    - "8000:8000"

  env_file:
    - .env

  depends_on:
    - db
    - redis

  networks:
    - app-network

db:

  image: postgres:16-alpine

  container_name: postgres

  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: root
    POSTGRES_DB: mes_db

  ports:
    - "5433:5433"

  volumes:
    - pgdata:/var/lib/postgresql/data

  networks:
    - app-network

  command: -p 5433

redis:

  build: ./redis

  container_name: redis

```

```
command: redis-server /usr/local/etc/redis/redis.conf

volumes:
  - ./redis/redis.conf:/usr/local/etc/redis/redis.conf

ports:
  - "6677:6677"

networks:
  - app-network

volumes:
  pgdata:
  redis_data:
  certs:
  vhost:
  html:
  acme:

networks:
  app-network:
    driver: bridge
```

Таким образом, данная конфигурация полностью готова к развертке. Скопировав при помощи ssh или клонировав файлы репозитория проекта при помощи git clone на удаленный сервер, мы можем развернуть весь проект лишь при помощи одного выполнения команды `docker compose up -d --build`.

Со всеми файлами проекта можно ознакомиться в репозитории github

Изображение №1. QR/Ссылка на репозиторий github



https://github.com/AntSib/MES_db_project

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.

1. Официальный репозиторий nginx-проxy // GitHub URL:
<https://github.com/nginx-proxy/acme-companion> (дата обращения: 04.06.2026).
2. Погружаемся в Docker: Dockerfile и коммуникация между контейнерами // habr.com URL: <https://habr.com/ru/companies/infobox/articles/240623/> (дата обращения: 02.06.2026).
3. Используем Docker и не волнуемся о vendor-lock // habr.com URL:
<https://habr.com/ru/companies/infobox/articles/237405/> (дата обращения: 02.06.2026).
4. Containers in 2025: Docker vs. Podman for Modern Developers // Linux Journal URL:
<https://www.linuxjournal.com/content/containers-2025-docker-vs-podman-modern-developers> (дата обращения: 20.06).
5. 9 Docker Alternatives in 2025: Podman, Rancher & Containerd Compared // Uptrace URL:
https://uptrace.dev/comparisons/docker-alternatives#_3-kubernetes-with-cri-o (дата обращения: 20.06).
6. What are the 5 best alternatives to Docker? // IONOS URL:
<https://www.ionos.com/digitalguide/server/know-how/docker-alternatives-at-a-glance/> (дата обращения: 20.06).